The Design and Implementation of a Capacity-Variant Storage System

Ziyang Jiao and Xiangqun Zhang, Syracuse University;
Hojin Shin and Jongmoo Choi, Dankook University; Bryan S. Kim, Syracuse University

2026. 08. 25

Presentation by Sung Jinwook, Jung Huichan
a011214@dankook.ac.kr

DANKOOK UNIVERSITY

Dankook University
System Software Laboratory

Contents

1.1. Problems of Wear-leveling

- CVSS is proposed to Resolve Fail-slow Problem of Wear-leveling

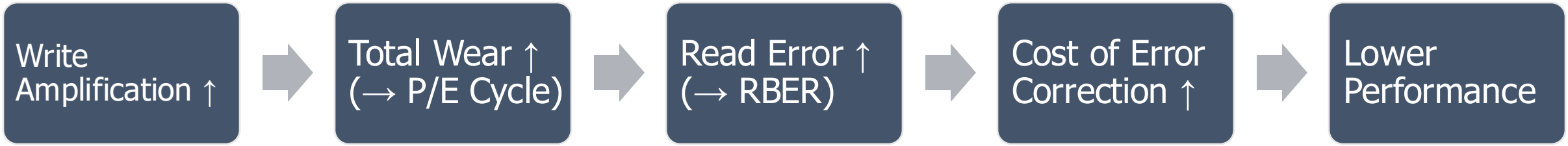
- What is Wear-leveling?
  - Flash Memory Blocks in SSD is Managed by Wearing out at the Similar Speed
    → To Preserve Overall Capacity of SSD

1.1. Problems of Wear-leveling

- 2 Policy of Wear-leveling
  - Static WL
    - Relocates Data Periodically Although it's Cold Data
    - Causes Write Amplification (Additional Write that Host did NOT Requested)
  - Dynamic WL
    - Executed Simultaneously with GC
    - Causes Degradation of Cleaning Efficiency (GC Also Have To Consider Wear State)
      → Causes Write Amplification

1.1. Problems of Wear-leveling

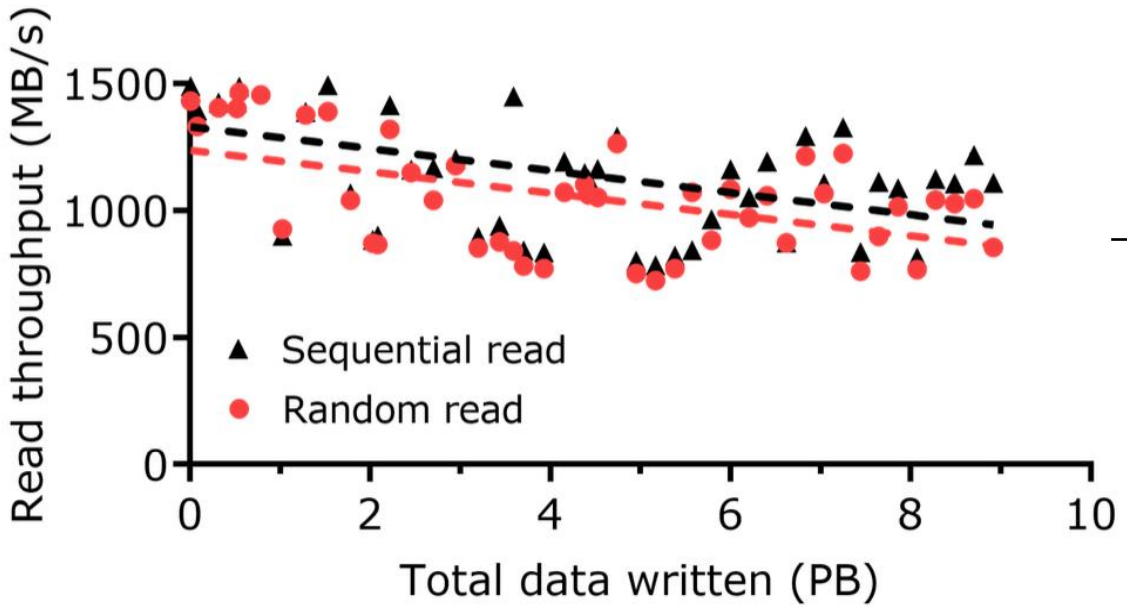
- Fail-slow Symptoms



- Internal Re-reads
- Preventive Re-writes

1.1. Problems of Wear-leveling

- SSD Performance Degradation due to Wear-out
  - 100TB Data Random Written per Every Day → Just Read
  - Proved that Fail-slow is Usual Symptom



→ After 9PBs are Recorded,
Sequential Reads 38% ↓
Random Reads 37% ↓

1.1. Problems of Wear-leveling

- Why Wear-leveling is Adopted
  - Fixed Capacity Feature in HDD is Inherited to SSD
    (HDD is Fail at the Device-level)
  - If Too Many Bad Blocks in SSD are Generated,
    Then SSD can NOT Maintain Logical Capacity Initially Promised to the FS
  - Wear-leveling to Preserve Fixed Capacity is Essential
    Even Though Additional Overhead Occurs

1.2. Proposal of Capacity-Variant SS

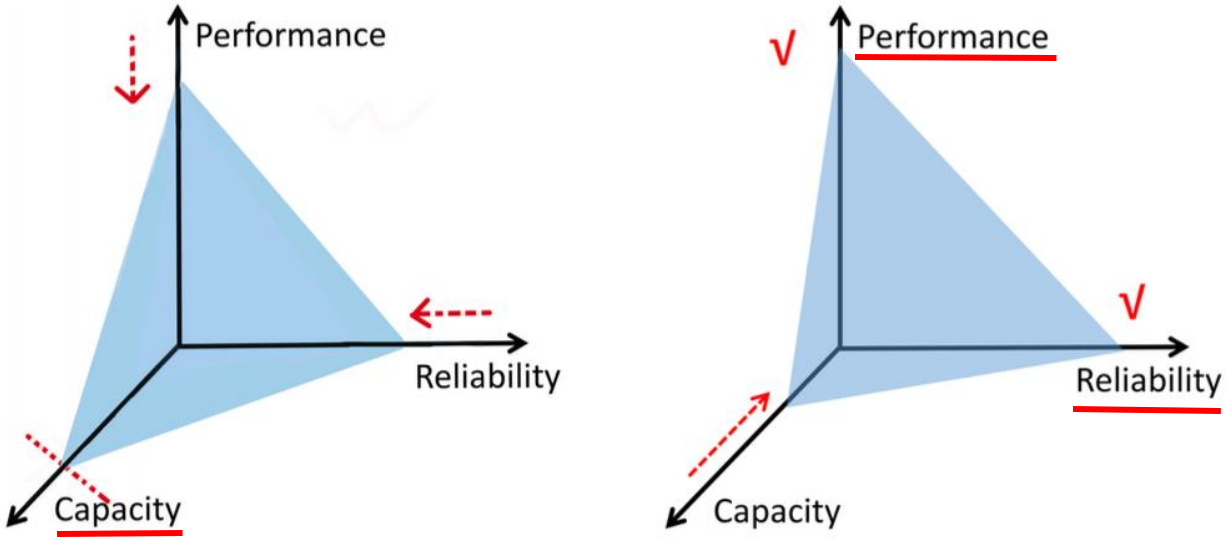
- Proposing Capacity-Variant Storage System
  - HDD is Fail at the Device-level (= Fail-stop)
    But SSD is Fail at the Block-level (= Fail-partial)
  - We do Not Need to Apply Fail-stop in SSD
    → Focus to Wear Older Blocks Even Faster

1.2. Proposal of Capacity-Variant SS

- 3-levels of Capacity-Variant Storage System
  1) Wear-focusing of CV-SSD
     - Give Up Wear-leveling → Shrinks Capacity Gradually
     - Another Most of Blocks Maintain Good Performance and Reliability
  2) Address Remapping of CV-FS
     - Reflects Physical Reduction of CV-SSD
     - Shrinks Logical Capacity Without Data Loss & Performance Degradation
  3) Orchestration of CV-Manager
     - Orchestrates Operations between CV-FS and CV-SSD

1.2. Proposal of Capacity-Variant SS

- Purpose of Capacity-Variant Storage System



Traditional Fixed-Capacity Storage System → Capacity-Variant Storage System

2. Design of CV-SSD

- Wear-focusing of CV-SSD
  - Does NOT Conduct Wear-leveling (before Degraded Mode)
  - Small Older Blocks are FIRST Mapped Out → Shrinks Capacity Gradually
  - Another Most of Blocks Maintain Good Performance and Reliability

2.1. Block Status Considering Read Error

- Young Blocks
  - Relatively Low Erase Count AND Low Read Error Rate
- Middle-aged Blocks
  - Higher Read Errors → Needs More Cost of Error Correction
- Retired Blocks
  - Worn Out Physically OR Higher Read Error Rate than Threshold (dft. 5%)
  - Excluded from Storing Data

2.2.1. Policy & Case of Data Allocation

- CV-SSD First Allocates Host Write(Data Written by Host) to Middle-aged Block
- When Write-intensive Data is Allocated to Block
  - Write-intensive Data : Often Overwrites (→ Host Write)
    … Changed Frequently
  - Type I) Write-intensive Data → Middle-aged Block
    … Wear-focusing (Ideal)
  - Type II) Write-intensive Data → Young Block
    … Invalid Ratio ↑ (No Problem with Cleaning Efficiency)

2.2.2. Problem Case of Data Allocation

- CV-SSD First Allocates GC Write(Relocated Data Written by GC) to Young Block
- When Read-intensive Data is Allocated to Block (With Traditional GC)
  - Read-intensive Data : Little Overwrites (→ Host Write)
    … NOT Changed Without Host Write
  - Type III) Read-intensive Data → Middle-aged Block
    … Read Error ↑ (Pay Attention)
  - Type IV) Read-intensive Data → Young Block
    … Wear-focusing (Ideal)

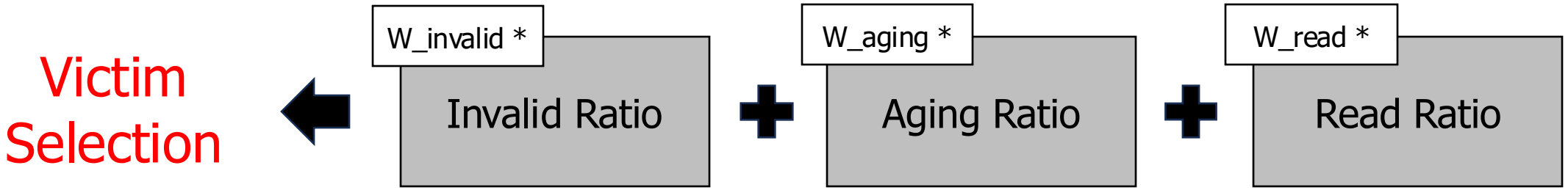
2.3.1. Changing GC Victim Selection

- We Have To Move

  Read-intensive Data in Middle-aged Block to Younger Block

  … To Decrease Error Correction Overhead

  → Consider Read Ratio when Garbage Collection



2.3.2. Factors of Selecting Victim

- Invalid Ratio
  - # of Invalid Pages ÷ # of (Invalid+Valid) Pages
  - Traditional Standard of GC Victim Selection
- Aging Ratio
  - Erase Count ÷ Endurance(Limit of P/E Cycle)
  - The Older, the More Selected → Opposite to Wear-leveling
- Read Ratio
  - # of Host Read Requests ÷ # of Max Host Read Req. Block of All of Active Blocks
  - 'How Intensively Being Read' is Indicated to 0 ~ 1

2.4. Mode to Preserve Minimum Capacity

- Degraded Mode
  - If Remained Physical Capacity Gets Too Low?
    → User Data is Endangered to Sudden Loss
  - Trigger Conditions
    1) EOP Rate(Effective OP) < OP Rate(Over-provisioning)
       - EOP = (Physical Capacity – Utilization) ÷ Utilization
       - Utilization = Total Capacity of Valid User Data (Information Provided by CV-FS)
       - If OP = 20%, Utilization = 80GB, Physical Capacity Changes 100GB to 90GB, EOP Changes 25% to 12.5% → Degraded Mode Activated
    2) Remained Physical Capacity < Watermark that User Configured

2.4. Mode to Preserve Minimum Capacity

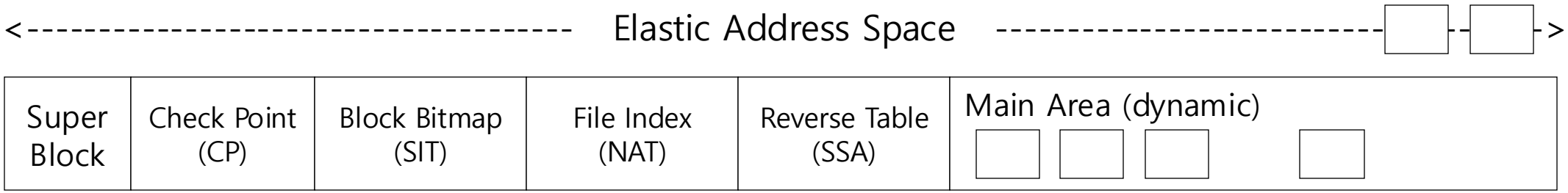
- Degraded Mode
  - Changes Compared to Normal Condition

	Normal	Degraded Mode
Block Allocation	Host Write → Middle-aged Blk GC Write → Young Blk	Host Write → Young Blk GC Write → Middle-aged Blk
GC Policy	Victim Score (Considers Invalid Ratio, Aging Ratio, Read Ratio)	Adjusted Victim Score (Considers Invalid Ratio, Aging Ratio)

  - Allows that Young & More Invalid Block Get GC-ed Prior to Older Block
  - Wearing Status Can Distributed More Evenly
    → Can Preserve Mimimum Physical Capacity for User

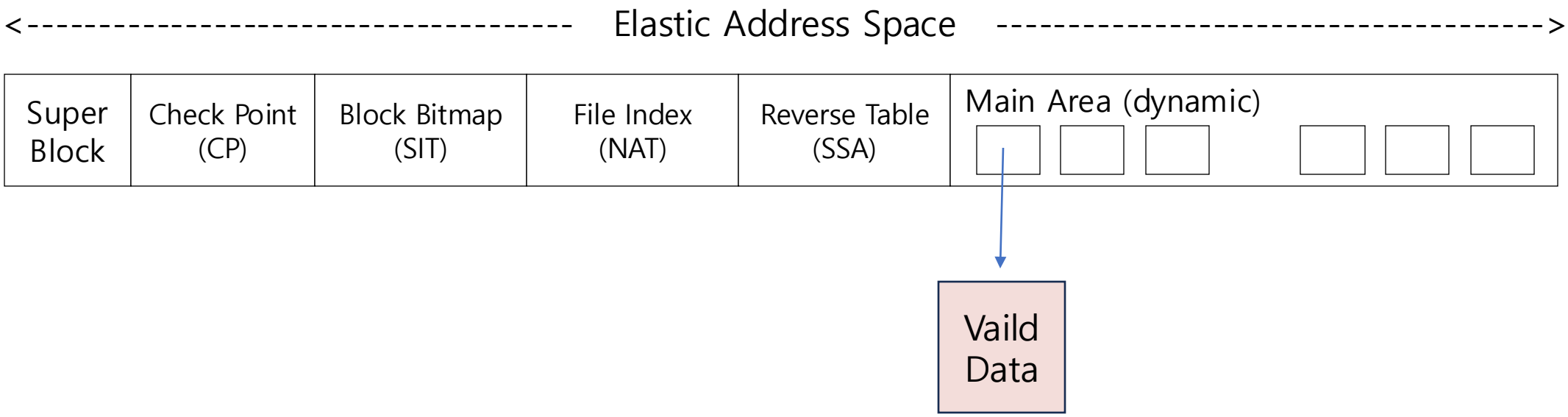
3. Design of CV-FS

- **Why CV-FS?**
  - **Log-strurctured file system**
    - Data management
    - Address remapping
  - **Elastic logical partition**
  - **Dynamically adjusts the logical partition as the SSD ages**



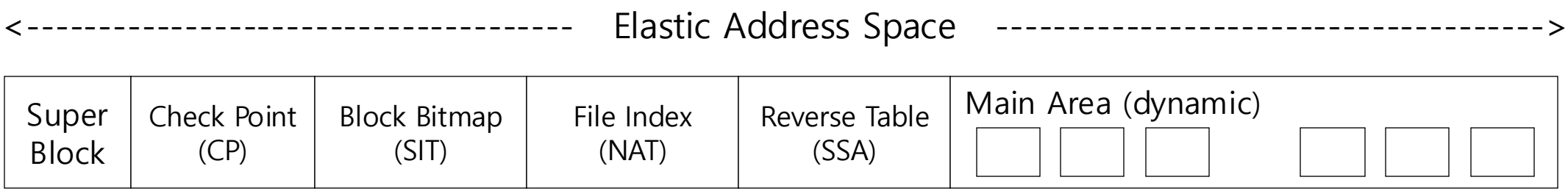
3. Design of CV-FS

- **CV-FS Requirements**
  - Data Consistency in capacity shrink



3. Design of CV-FS

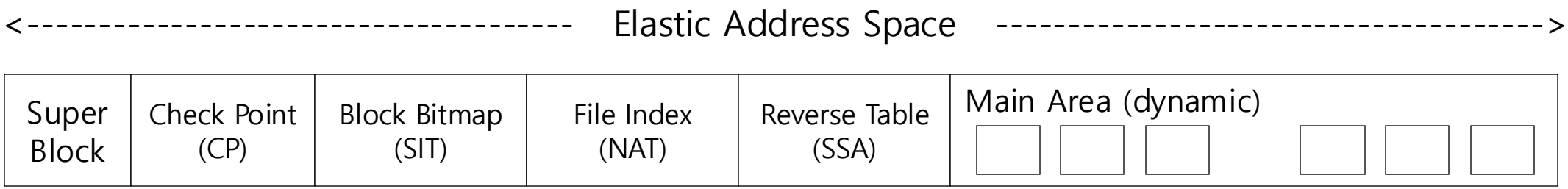
- **CV-FS Requirements**
  - Data Consistency
  - Online Adjustment (not unmount)



Super Block	Check Point (CP)	Block Bitmap (SIT)	File Index (NAT)	Reverse Table (SSA)	Main Area (dynamic)

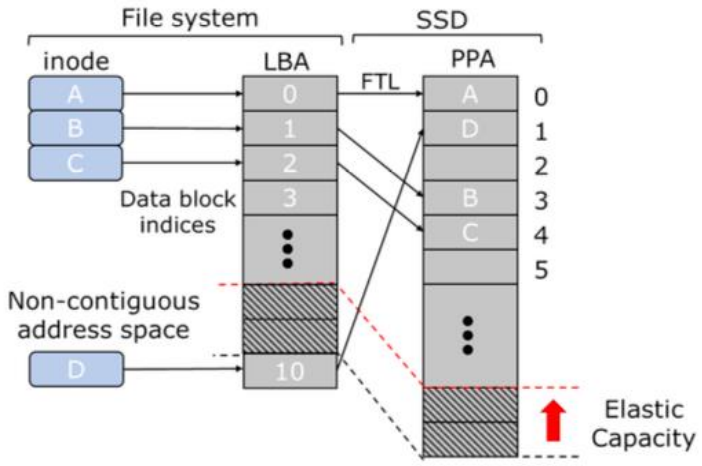
3. Design of CV-FS

- **CV-FS Requirements**
  - Data Consistency
  - Online Adjustment
  - Low Overhead(WAF, Lifetime)

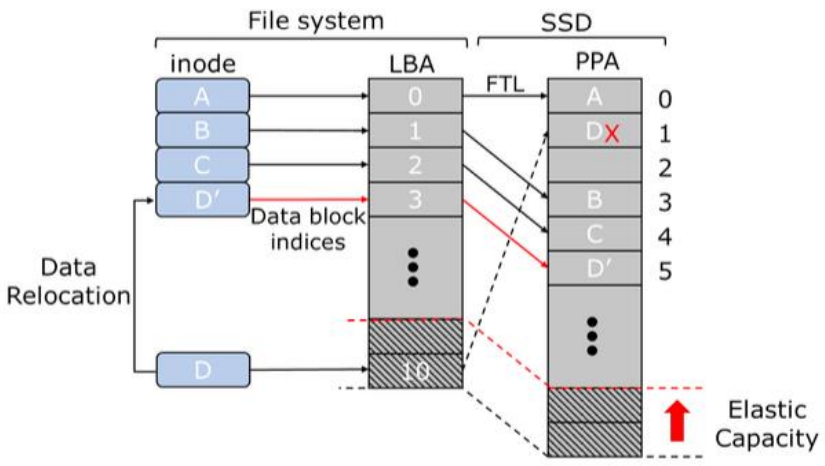


3. Design of CV-FS

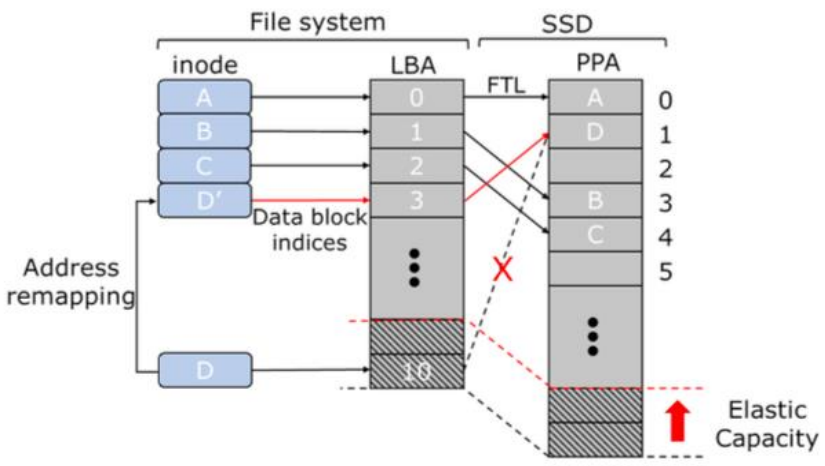
- **File System Designs for Capacity Variance**



(a) Non-contiguous address space



(b) Data relocation

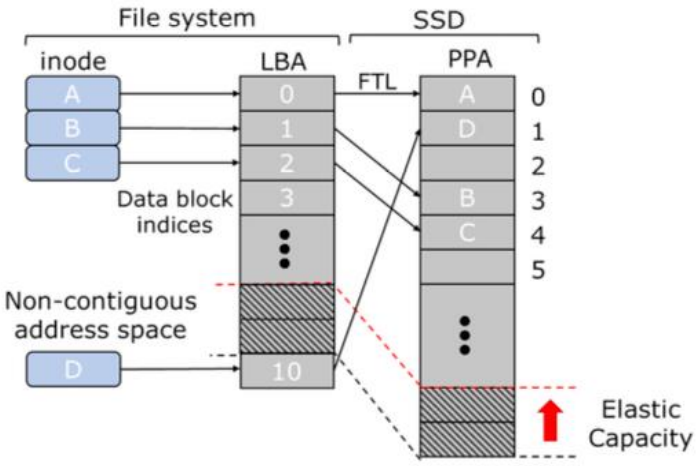


(c) Address remapping

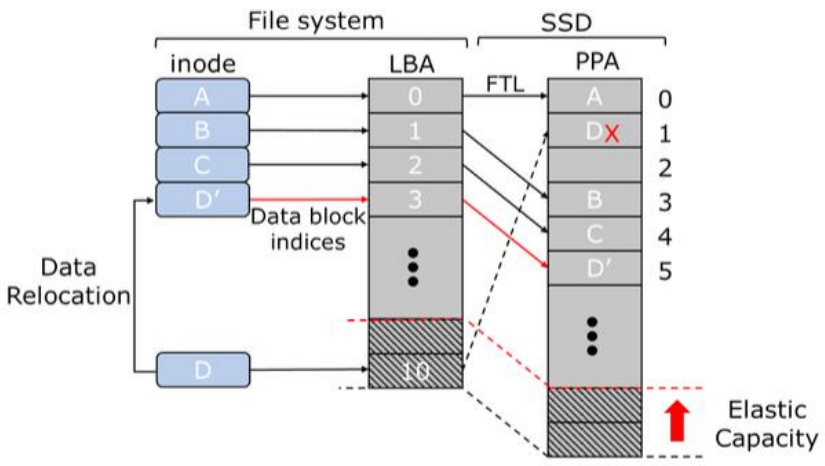
- minimal upfront costs
- Fragmentation
- Increase LFSs cleaning overhead

3. Design of CV-FS

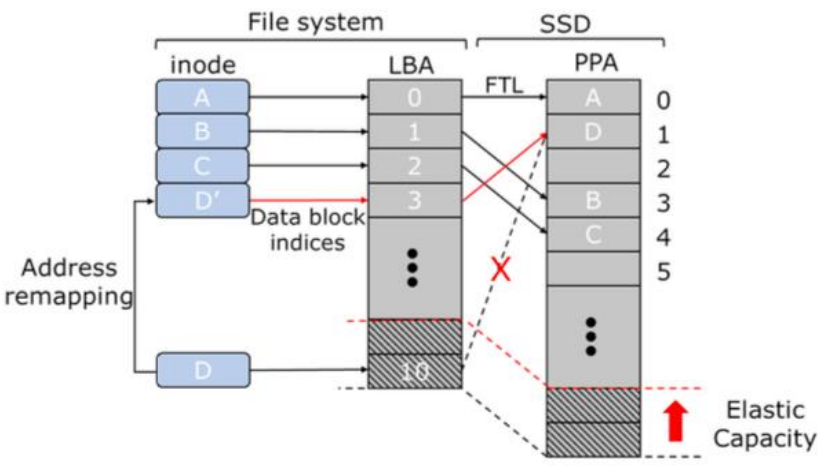
- **File System Designs for Capacity Variance**



(a) Non-contiguous address space



(b) Data relocation



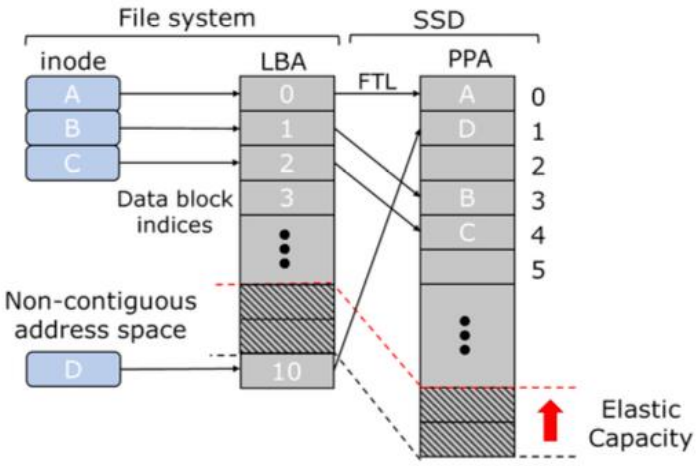
(c) Address remapping

- minimal upfront costs
- Fragmentation
- Increase LFSs cleaning overhead

- Maintain address space
- Exert additional write on the SSD

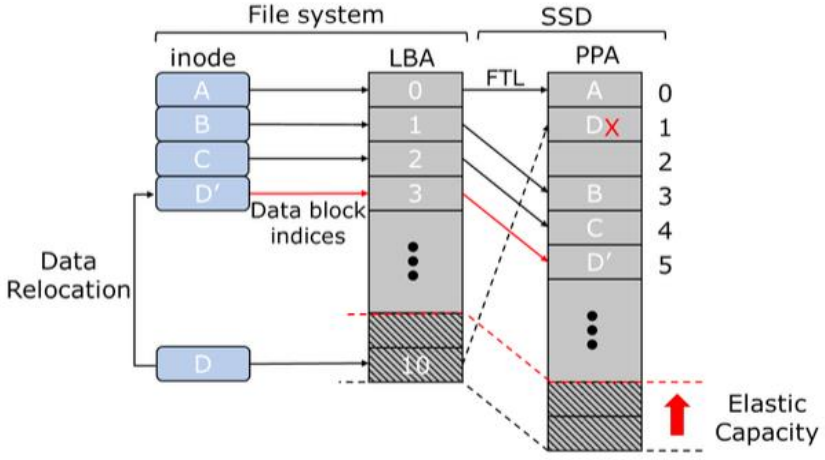
3. Design of CV-FS

- **File System Designs for Capacity Variance**



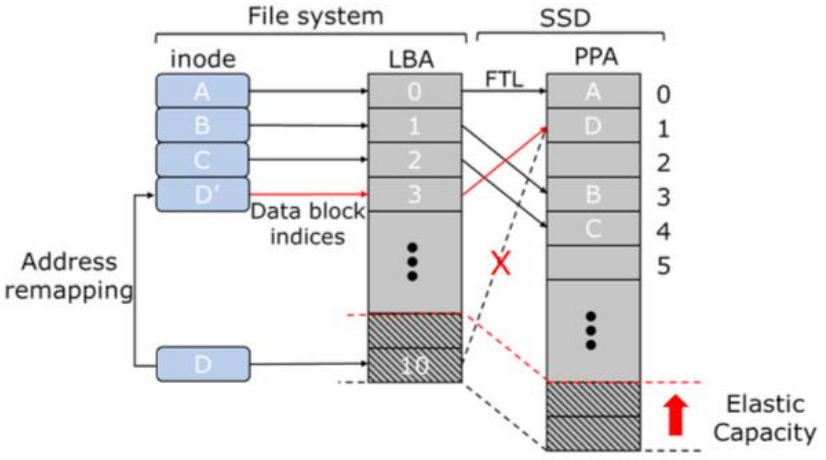
(a) Non-contiguous address space

- minimal upfront costs
- Fragmentation
- Increase LFSs cleaning overhead



(b) Data relocation

- Maintain address space
- Exert additional write on the SSD

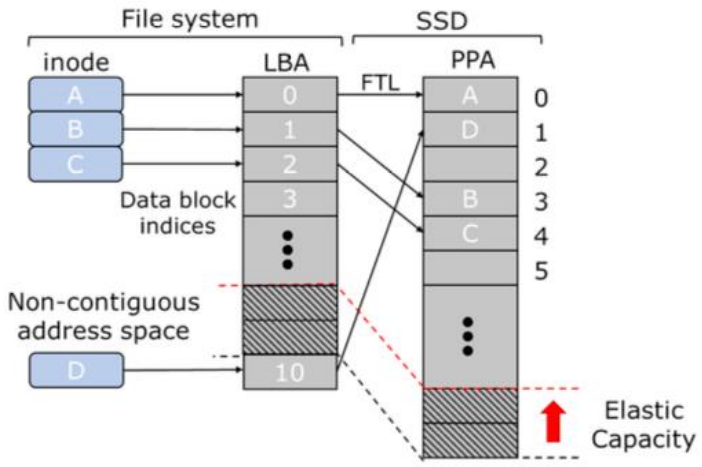


(c) Address remapping

- Maintain address space
- Negligible system overhead

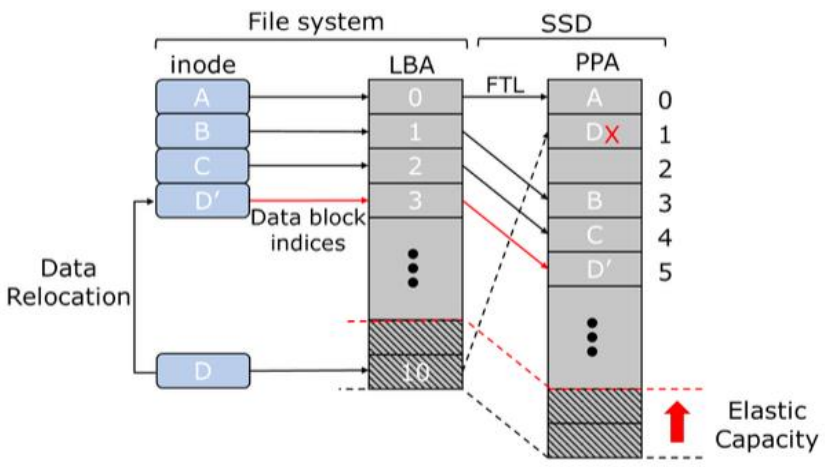
3. Design of CV-FS

- **File System Designs for Capacity Variance**



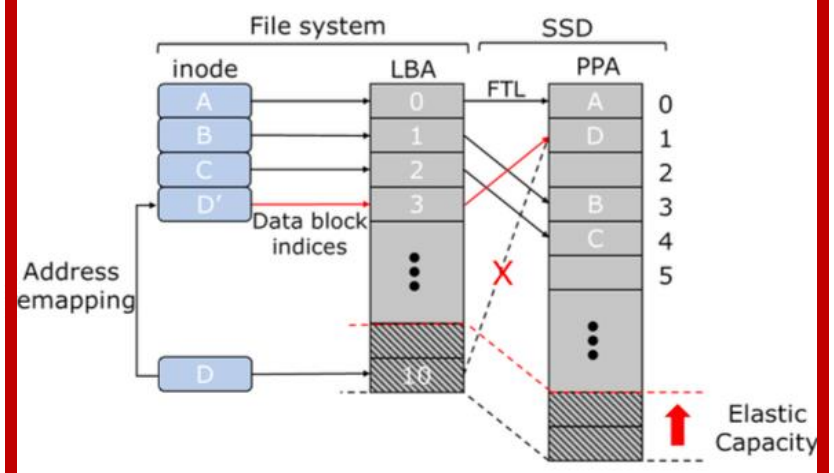
(a) Non-contiguous address space

- minimal upfront costs
- Fragmentation
- Increase LFSs cleaning overhead



(b) Data relocation

- Maintain address space
- Exert additional write on the SSD

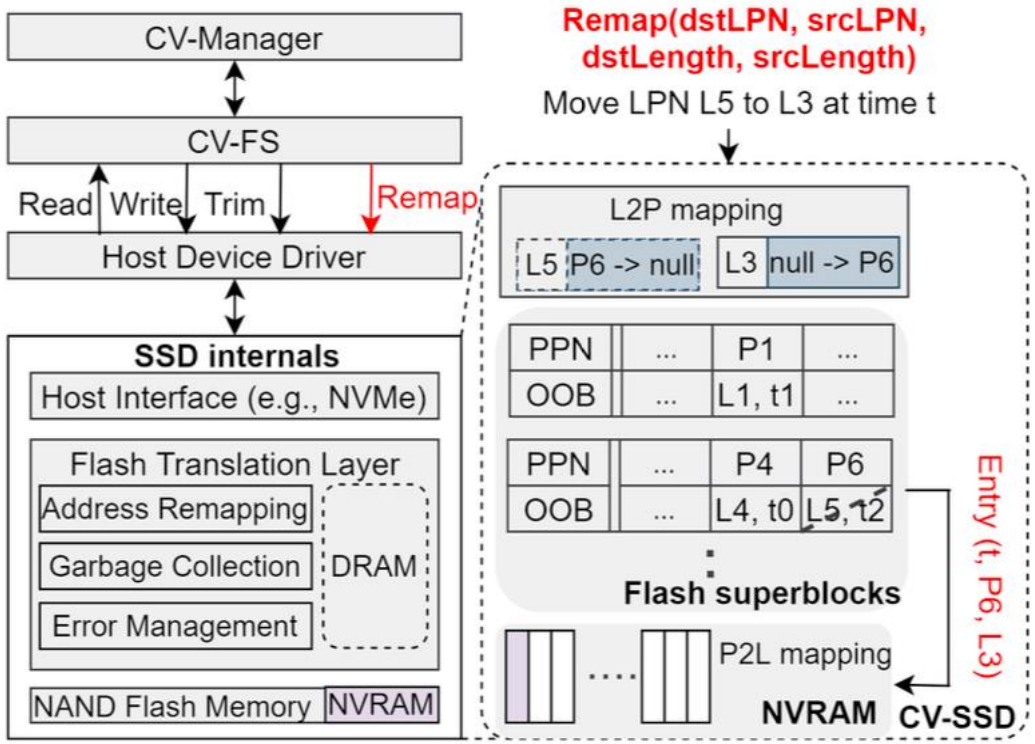


(c) Address remapping

- Maintain address space
- Negligible system overhead

3. Design of CV-FS

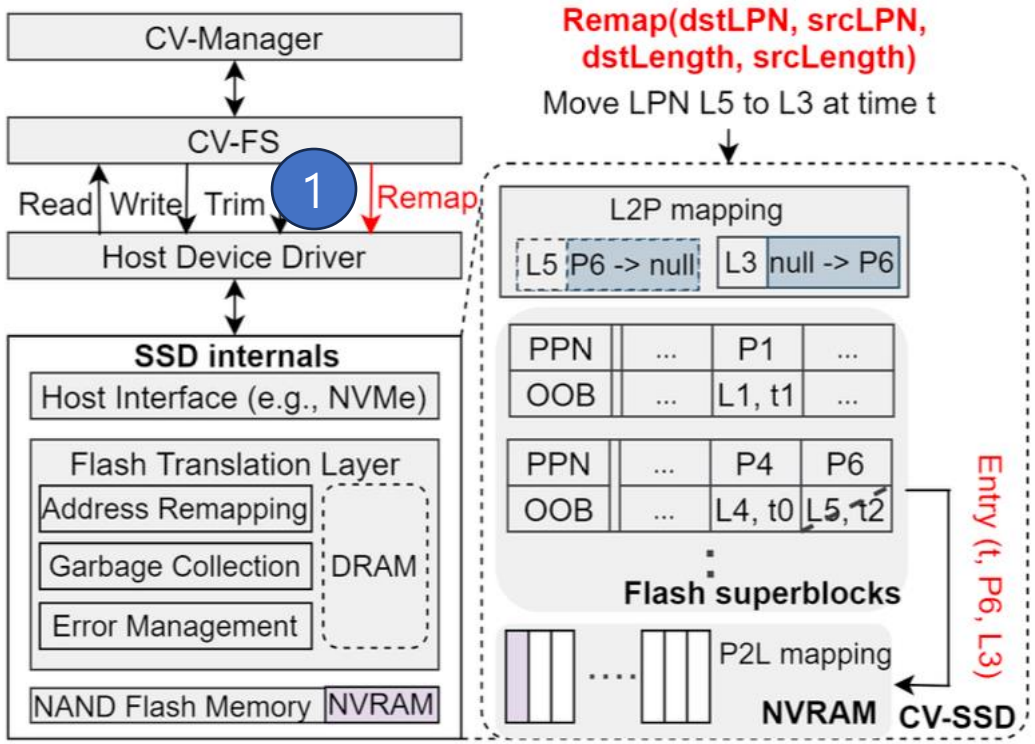
- **Interface Changes for Capacity Variance**
  - Conventional SSD interface
    - Read / write / trim
  - REMAP (dstLPN, srcLPN, dstLength,srcLength)
    - dstLPN: new logical page number
    - srcLPN: old logical page number
    - dstLength / srcLength
      - Length informaction



3. Design of CV-FS

- **Interface Changes for Capacity Variance**
  - REMAP (L3, L5, 1, 1)

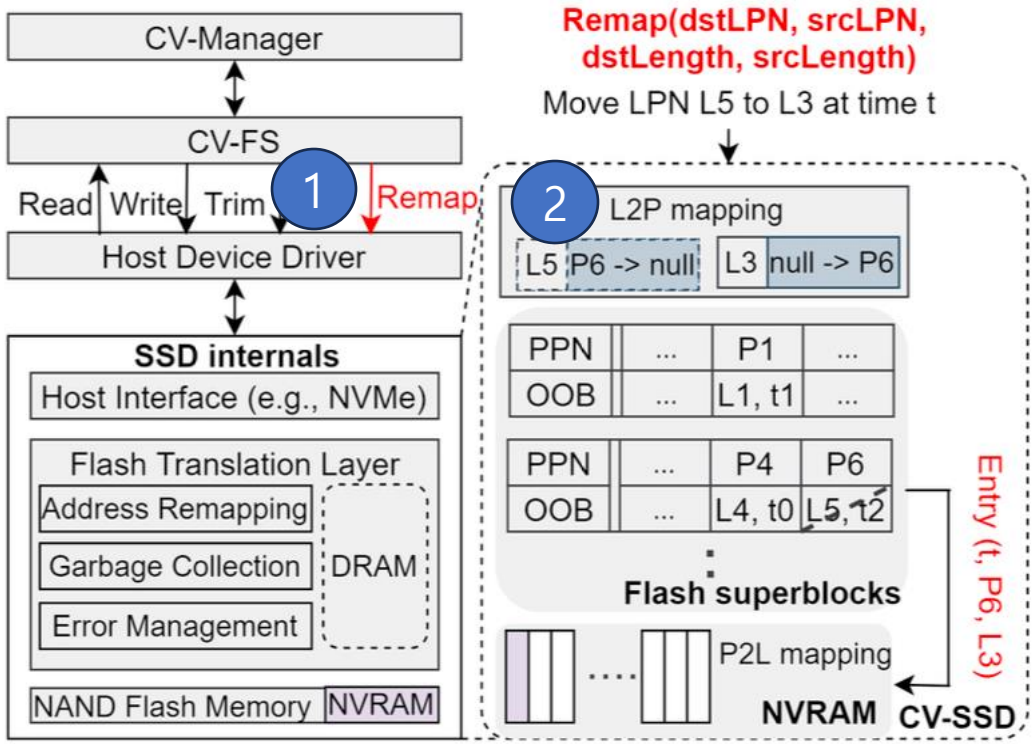
1. CV-FS → SSD: REMAP 요청 전달



3. Design of CV-FS

- **Interface Changes for Capacity Variance**
  - REMAP (L3, L5, 1, 1)

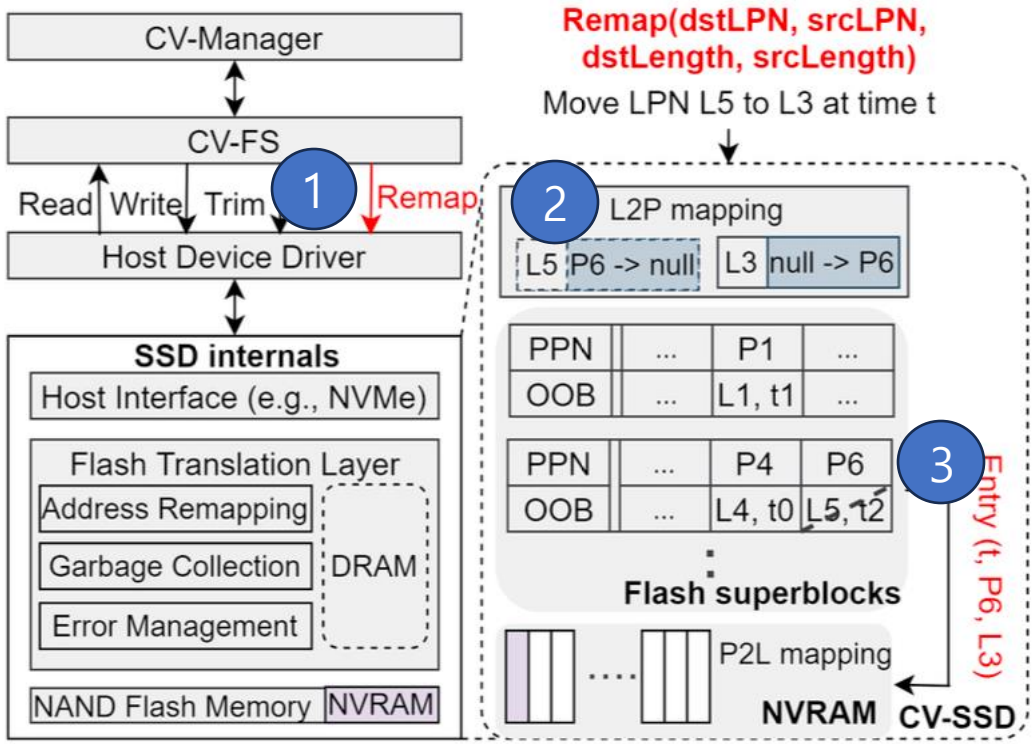
1. CV-FS → SSD REMAP: 요청 전달
2. Access L2P mapping: L5 → P6



3. Design of CV-FS

- **Interface Changes for Capacity Variance**
  - REMAP (L3, L5, 1, 1)

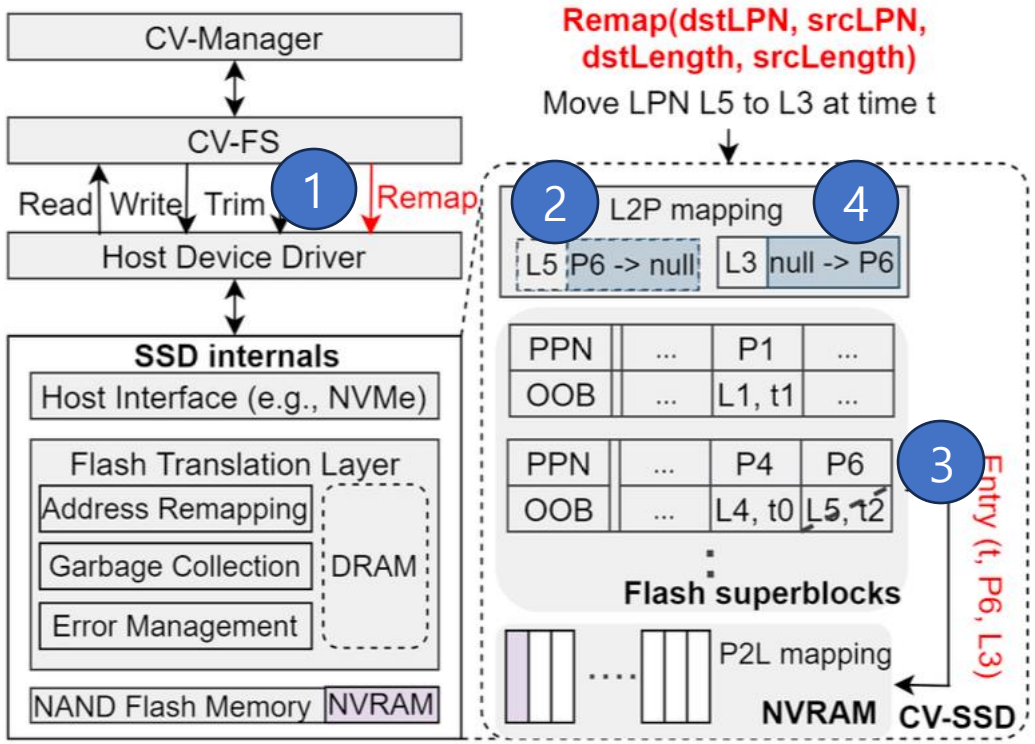
1. CV-FS → SSD REMAP: 요청 전달
2. Access L2P mapping: L5 → P6
3. Check OOB of P6: validation



3. Design of CV-FS

- **Interface Changes for Capacity Variance**
  - REMAP (L3, L5, 1, 1)

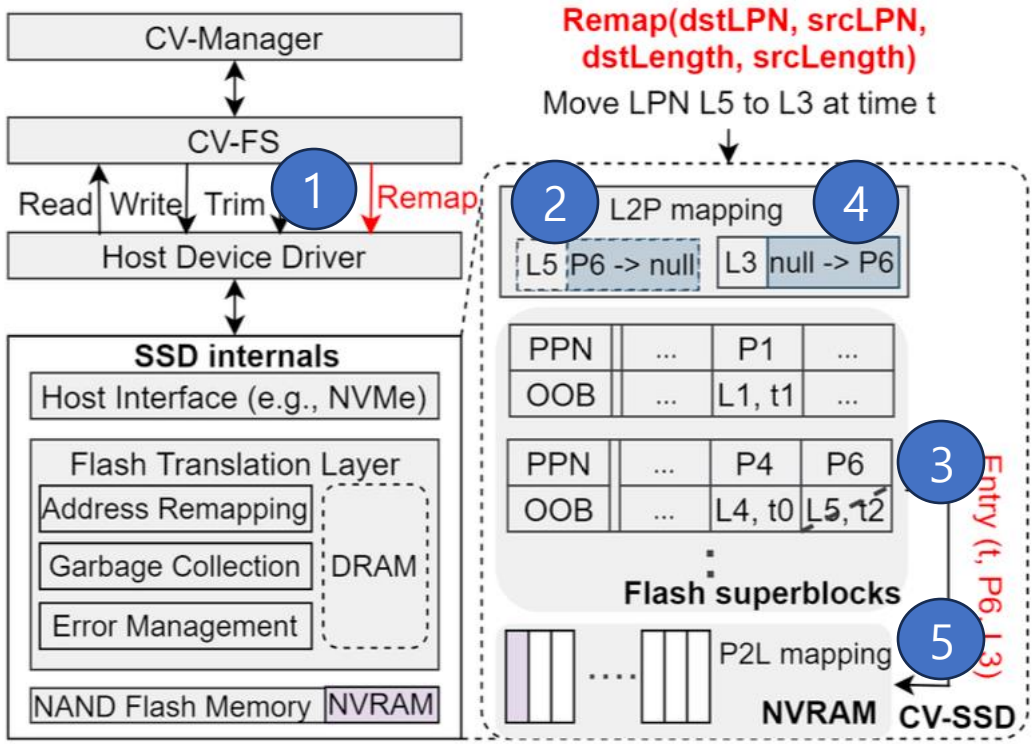
1. CV-FS → SSD REMAP: 요청 전달
2. Access L2P mapping: L5 → P6
3. Check OOB of P6: validation
4. Update L2P mapping: L3 → P6



3. Design of CV-FS

- **Interface Changes for Capacity Variance**
  - REMAP (L3, L5, 1, 1)

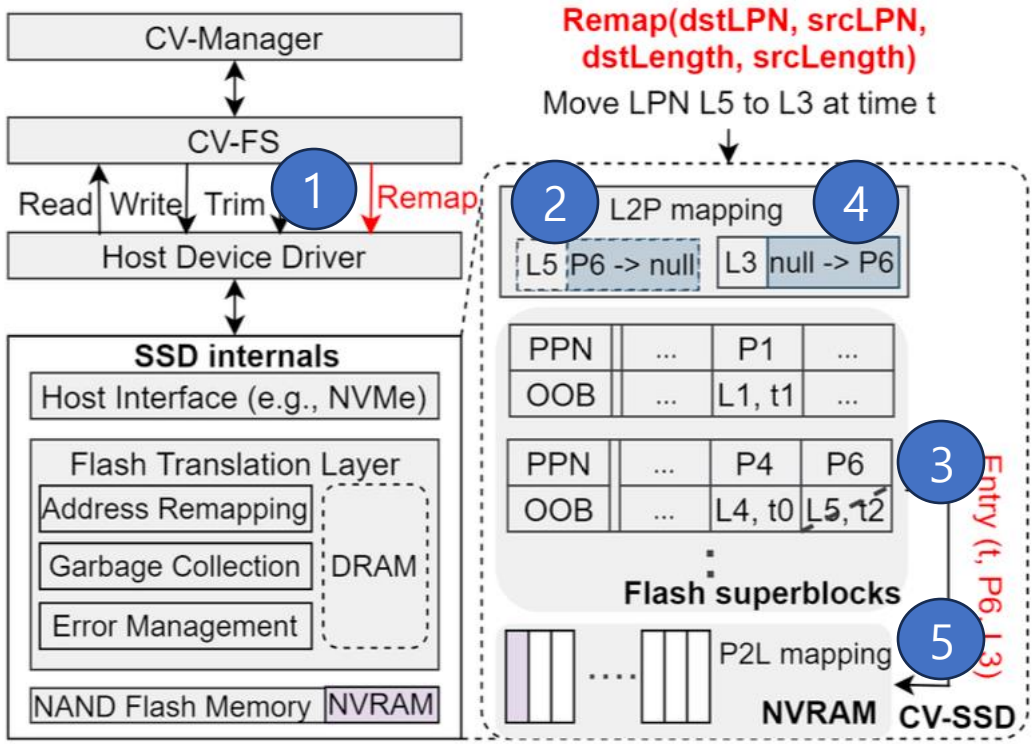
1. CV-FS → SSD REMAP: 요청 전달
2. Access L2P mapping: L5 → P6
3. Check OOB of P6: validation
4. Update L2P mapping: L3 → P6
5. Update P2L mapping: P6 → L3



3. Design of CV-FS

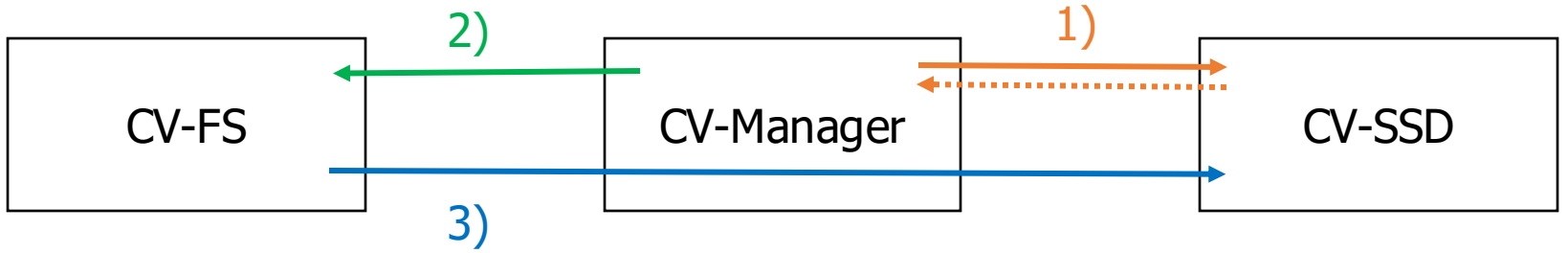
- **Interface Changes for Capacity Variance**
  - REMAP (L3, L5, 1, 1)

- **No physical data movement in NAND flash**
- **Only the address mapping is updated**
- **Supports capacity variance without additional NAND writes.**



4. Design of CV-Manager

- User-level Orchestrator between CV-FS and CV-SSD
  1) Monitors Physical Aging Status of CV-SSD
  2) Requests Logical Capacity Reduction to CV-FS
  3) Delivers CV-FS's Reduction Information to CV-SSD



- User can Configure Settings
  about Capacity, Performance and Reliability

5. Implementation of CVSS

- To Compare TrSS(Traditional SS) and CVSS

	TrSS	CVSS
Block Allocation	Youngest Blk First	Host Write → Middle-aged Blk GC Write → Young Blk
GC Policy	Greedy (Considers Invalid Ratio)	Victim Score (Considers Invalid Ratio, Aging Ratio, Read Ratio)
Wear-leveling	Progressive WL*	X (Wear-focusing)
More Aggressive Discard Policy* (Both Equals)		

- Progressive WL : As Erase Count Get Higher, Wear-leveling Get More Frequent (Categorized as Static WL)
- Aggressive Discard Policy : To Decrease Semantic Gap → Increase Cleaning Efficiency

5. Implementation of CVSS

- To Compare TrSS(Traditional SS) and CvSS
  - CV-FS
    - Based on Log-structured F2FS
  - CV-SSD
    - Based on FEMU
    - Implemented Read Error Calculating Model (= RBER, Raw Bit Error Rate) Varying on P/E Cycles, Time that Data Stored and Read Frequency

5. Implementation of CVSS

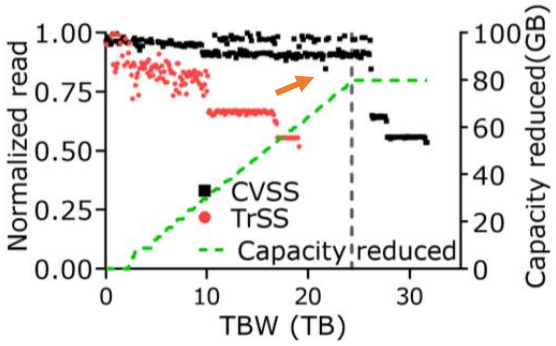
- To Compare TrSS(Traditional SS) and CvSS
  - Over-provisioning ≈ 7.37%
    - The Lower OP Rate is Set … The Faster Degraded Mode is Triggered
    - OP = Logical Capacity(120GiB) / Physical Capacity(128GiB)
  - Endurance = 300
    - The Limitation of P/E Cycle
    - The Lower Endurance is Set … The Faster Block Get Aged

6. Evaluation

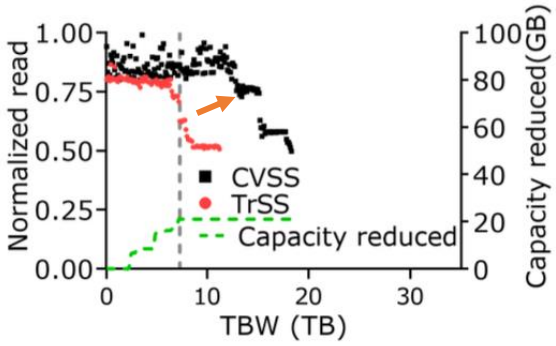
- 3 Questions Should be Evaluated
  1) Can CVSS Maintain Performance while Storage Device Get Aged?
  2) Can CVSS Extend Device Lifetime
     under Various Performance Requirements?
  3) What are the Tradeoffs in CVSS Design?

6.1. Evaluation of Maintaining Performance

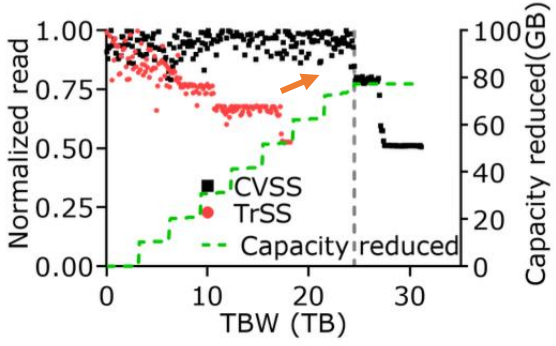
- Experiment that Recording Read Thoroughput
  while Continuously Generating 16KB Read/Write Requests in FIO
  - Green Line = Reduced Capacity of Device
  - Grey Vertical Line = Moment that CVSS Changes to Degraded Mode



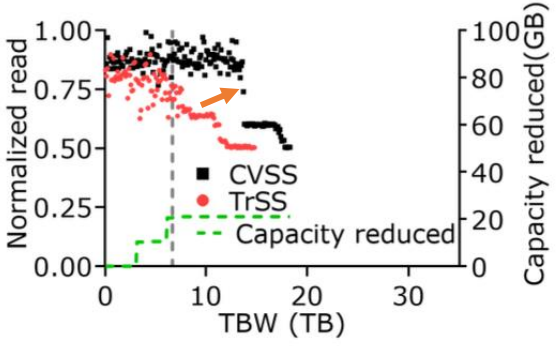
(a) Zipfian (utilization 30%)



(b) Zipfian (utilization 70%)



(a) Random (utilization 30%)

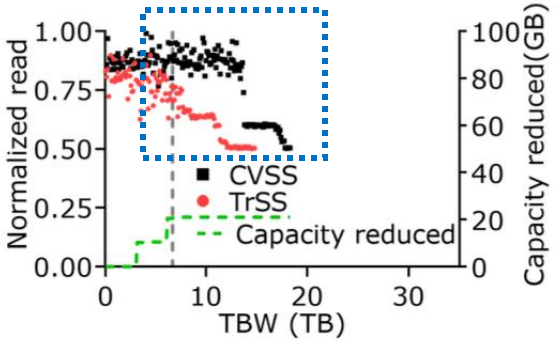


(b) Random (utilization 70%)

Result 1) Both of Read Throuthput : TrSS → CVSS 60~70% ↑ (under the Same Amount of Host Writes)

6.1. Evaluation of Maintaining Performance

- Experiment that Recording Read Thoroughput
  while Continuously Generating 16KB Read/Write Requests in FIO
  - Green Line = Reduced Capacity of Device
  - Grey Vertical Line = Moment that CVSS Changes to Degraded Mode

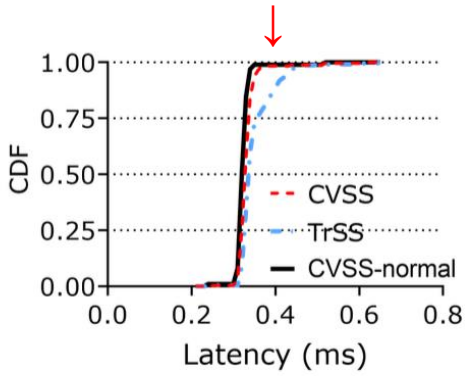


(b) Random (utilization 70%)

Result 2) CVSS After Degraded Mode Indicates Better Performance than TrSS (by Random, util. 70%)

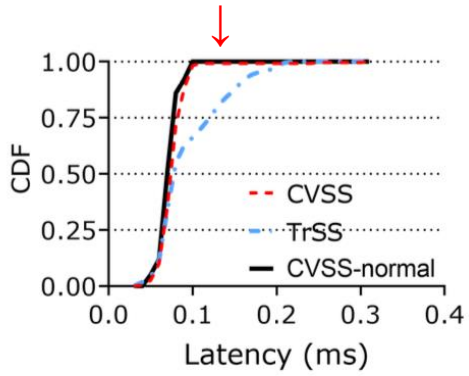
6.1. Evaluation of Maintaining Performance

- Experiment that Measuring Cumulative Distribution of Latency in FileBench After 100GB of Cumulative Random Writes
  - CVSS-normal = CVSS before Degraded Mode, CVSS = Total Lifetime of CVSS



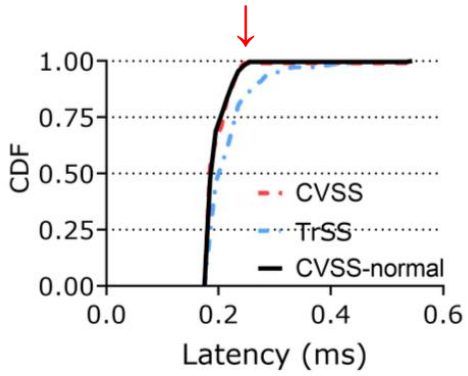
(a) Fileserver

→ Latency :
TrSS → CVSS 8% ↓



(b) Netsfs

→ Latency :
TrSS → CVSS 24% ↓
CVSS-normal 32% ↓

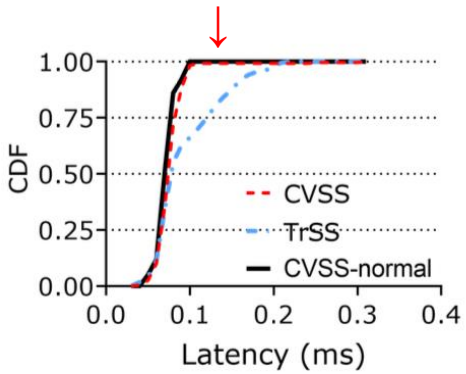


(c) Varmail

→ Latency :
TrSS → CVSS 12% ↓

6.1. Evaluation of Maintaining Performance

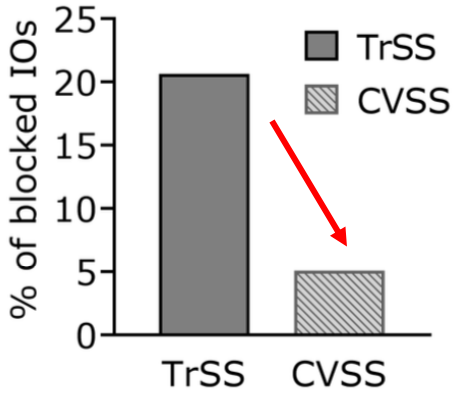
- Experiment that Measuring Cumulative Distribution of Latency After 100GB of Cumulative Random Writes
  - CVSS-normal = CVSS before Degraded Mode, CVSS = Total Lifetime of CVSS



(b) Netsfs

→ Latency :
TrSS → CVSS 24% ↓
CVSS-normal 32% ↓

Why? Read-Modify-Write Pattern of Netsfs → Sensitive to Aging Status of Device

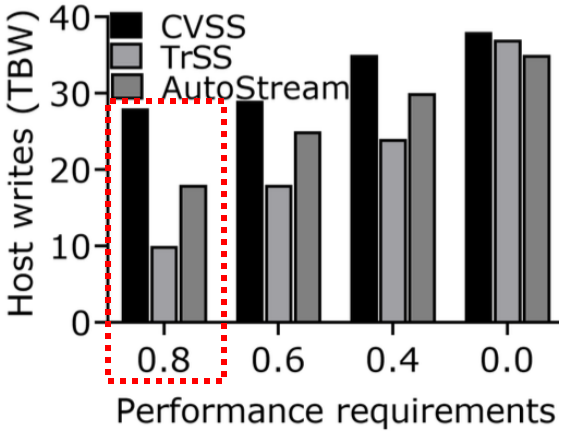


(d) Host I/Os blocked by error handling under varmail

→ Host I/O by Read Retry :
TrSS → CVSS 4× ↓

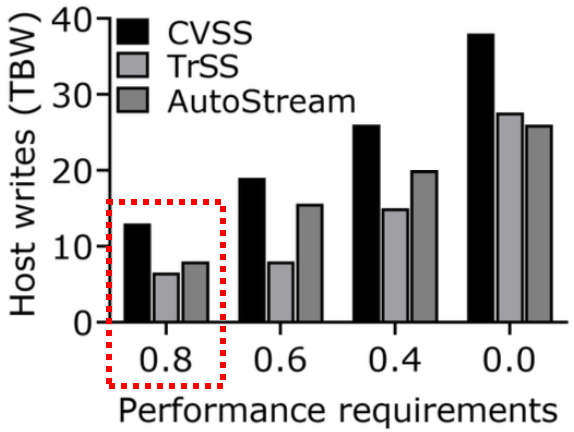
6.2. Evaluation of Extending Lifetime

- Experiment that Recording Total Written
  when Performance First Drops Below (80%) * Initial
  - Similar Result between Zipfian and Random Workload



(a) Zipfian (30% utilization)

Total Written when 0.8↓ :
TrSS → CVSS 180%↑
AutoStream → CVSS 55%↑

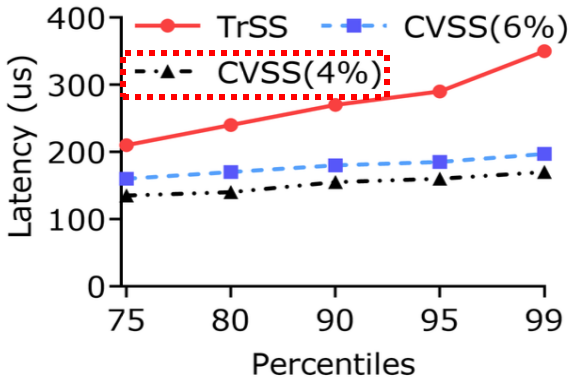


(b) Zipfian (70% utilization)

Total Written when 0.8↓ :
TrSS → CVSS 123%↑
AutoStream → CVSS 55%↑
(avg. of Zipfian & Random)

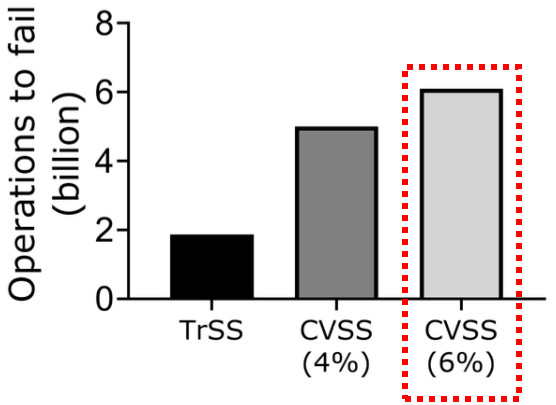
6.3. Evaluation of Trade-offs in CVSS

- Mapping-out Threshold in CVSS: Exceeds 4% vs 6% of RBER
  - Similar Result between YCSB-A and YCSB-F Workload



(a) Read latency (YCSB-A)

p99 Read Latency :
TrSS → CVSS(4%) 51%↓
TrSS → CVSS(6%) 44%↓



(c) Lifetime (YCSB-A)

Lifetime :
TrSS → CVSS(4%) 2.68×
TrSS → CVSS(6%) 3.27×
(avg. of YCSB-A & YCSB-F)

7. Conclusion

- Designed a cross-layer architecture across the file system, SSD, and management layer.
  - Links SSD physical capacity changes to filesystem logical capacity adjustment.
- Improved SSD lifetime and mitigated aging-induced performance degradation
- Achieved lower write amplification, improved performance compared to existing approaches

Thank you & Any Questions?

Appendix: CV-SSD without Wear-focusing

- More than Half of Blocks Get Ages Similarly
  → Danger of Sudden Capacity Loss

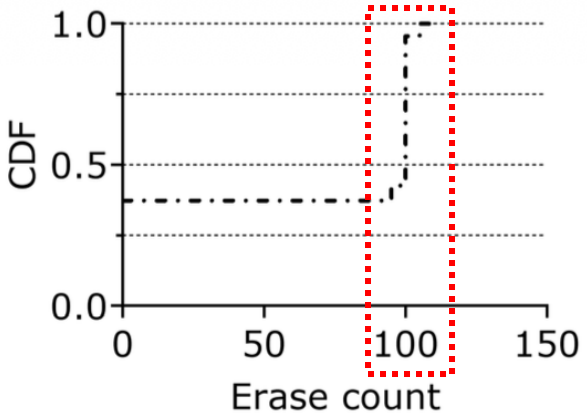


Figure 8: The wear distribution for a 256 GiB SSD under 100 iterations of MS-DTRS workload [29]. Traditional GC and block allocation policies cause a sudden capacity loss as too many blocks are equally aged.

Appendix: RBER Formula in CVSS

- Varying on P/E Cycles, Time (that Data Stored) and Read Frequency

$$
\begin{aligned}
&RBER(cycles, time, reads) \\
&\quad = \varepsilon + \alpha \cdot cycles^{k} && (wear) \\
&\qquad + \beta \cdot cycles^{m} \cdot time^{n} && (retention) \\
&\qquad + \gamma \cdot cycles^{p} \cdot reads^{q} && (disturbance)
\end{aligned}
\tag{2}
$$

Appendix: Parameters of GC Formula

- Configured Parameters
  - W_invalidity = 0.4
  - W_aging = 0.3
  - W_read = 0.3

$$Victim\ score = W_{invalidity} \cdot invalid\ ratio + W_{aging} \cdot aging\ ratio + W_{read} \cdot read\ ratio$$

- Experiment in Zipfian

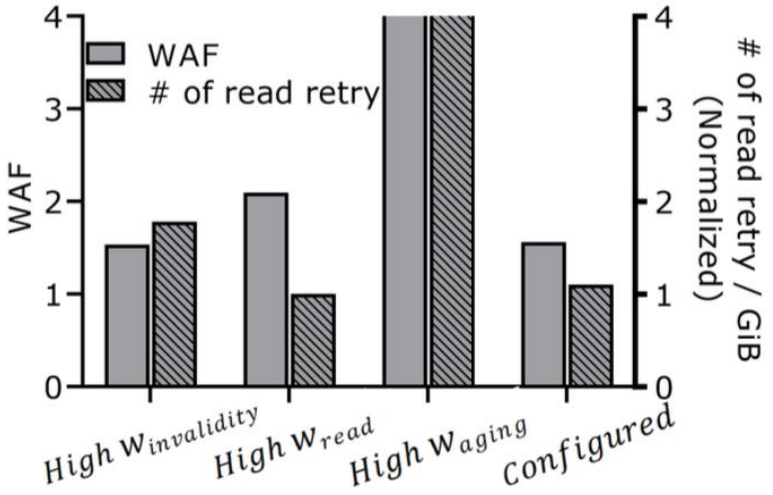


Figure 19: The WAF and read retries triggered under different weights used for GC formula.

Appendix: Workflow of CV-Manager

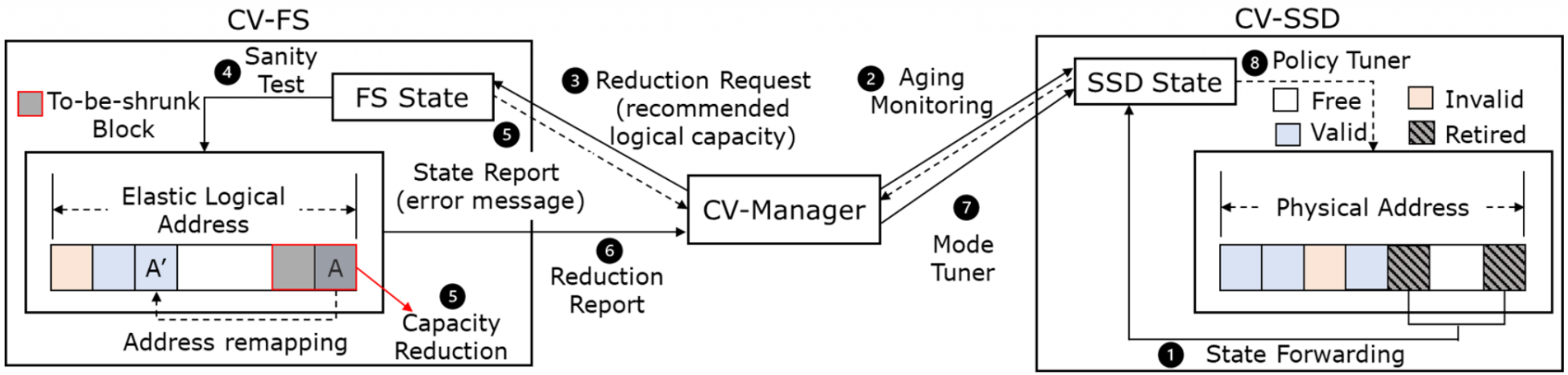


Figure 9: CV-manager design diagram. CV-manager monitors CV-SSD's aged state (Steps 1 and 2) and provides a recommended logical capacity to CV-FS (Step 3). After capacity reduction (Steps 4–6), CV-manager notifies CV-SSD (Step 7). The $CV_{degraded}$ mode will be triggered if the reduction fails (Step 8).

Appendix: Discussion

- Most Useful when I/O Performance is Bottlenecked
  But Spare Capacity Exists (ex. Facebook Haystack)